

画面設計

目次

1 静的配信 + 3 つの動的経路	4
2 サーバを挟む理由(素の http.server を使わない)	4
3 GET	6
4 POST(すべて書き込み・実行系)	6
5 安全設計	6
6 2 つの実装方式	8
7 出現条件の決定表	8
8 承認ウィジェットの構造	9
9 WBS(トップ)	10
10 ① 一斉レビュー表	10
11 バッチ実行状況ページ	10
11.1 設計の要点	11
11.2 残タスクの分類	11
11.3 ポーリング設計	11
12 配布形態による機能差	11
13 実行枠の排除	13

i この章の範囲と「正」

人が触れる画面(HTML)と、その裏側の HTTP エンドポイント・状態ファイル・描画タイミングを扱う。「どの画面に何が出るか」「その表示は誰が・いつ作るか」が主題。

挙動の正は [references/workflow.md](#)(機械が従う実行時規則)、操作手順の正は [MANUAL.md](#)(操作者向け)にあり、本章はそこから設計意図を解説する。フェーズそのものの仕様はパイプライン仕様、データの形はデータ仕様を参照。

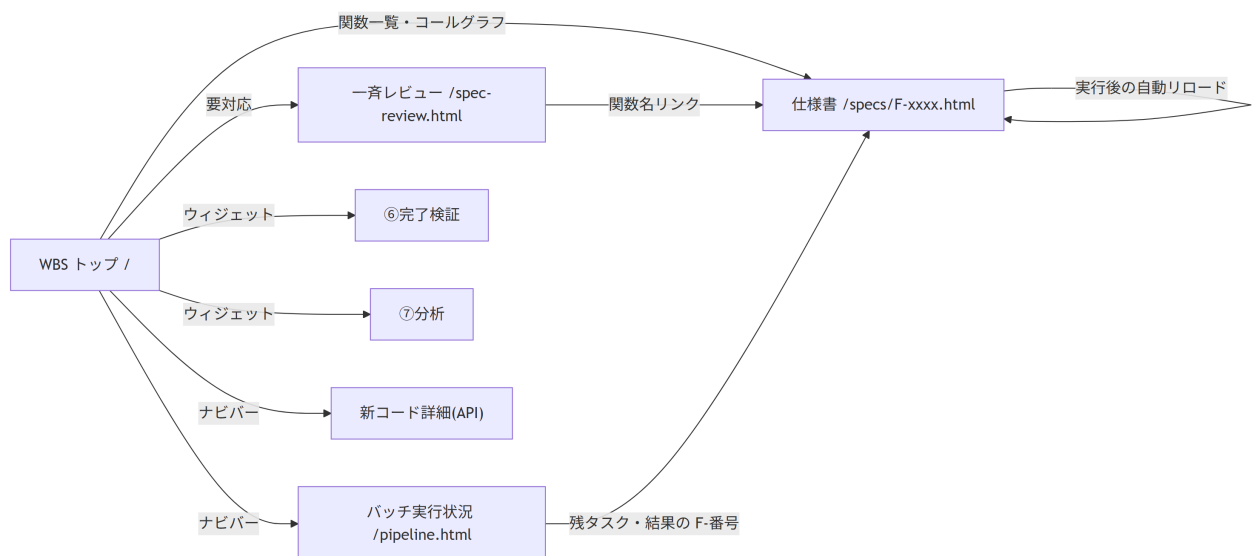
画面が担うもの

このシステムの人の仕事は4つ — トリガを引く・質問に答える・承認する・裁定する。画面はこの4つを「チャットに戻らずに」完結させるために存在する。逆に言えば、画面は進捗の閲覧装置ではなく操作装置として設計されている。

表 1

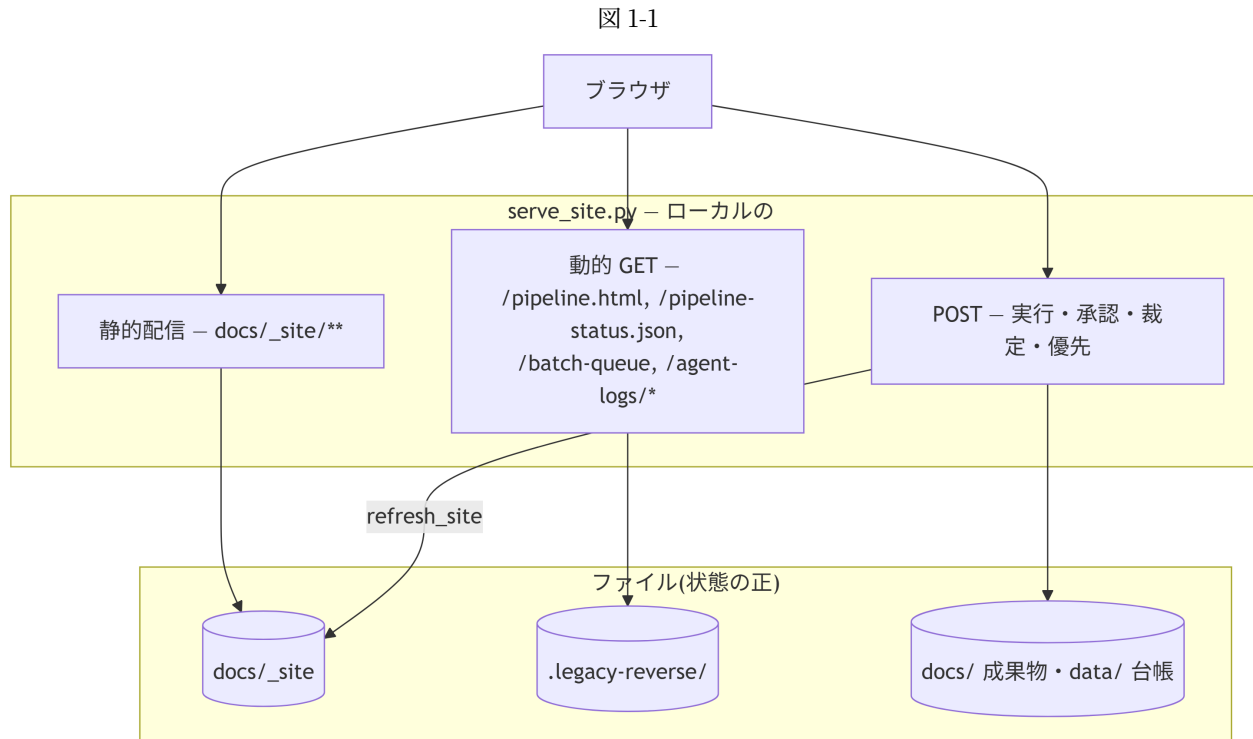
画面	URL	主な役割	生成方法
WBS(トップ)	/	進捗の全体像・要対応・次の一手。⑥⑦の起動	Quarto(ledger.py wbs が index.qmd を生成)
関数仕様書	/specs/F-xxxx.html	①の閲覧。関数の「ホーム」として①～⑤の実行・承認・裁定ボタンが集まる	Quarto(成果物 .md + ウィジェット焼き込み)
① 一斉レビュー	/spec-review.html	draft の仕様書をまとめて承認・修正依頼	Quarto(review_checks.py report が .md を生成)
バッチ実行状況	/pipeline.html	無人実行の運転席。状況把握・順番の割り込み・人待ちの解消	serve_site.py が動的配信(Quarto を通さない)
⑥ 完了検証	/completion-check.html	全関数の完了判定レポート	Quarto(ledger.py check が生成)
⑦ 分析 / 定量評価	/analysis.html ・ /quant.html	施策候補と実測データ	Quarto
新コード詳細 (API)	/api/index.html	④実装の docstring から生成した API リファレンス	Sphinx(別系統)

図 1



1 静的配信 + 3 つの動的経路

`serve_site.py` は Quarto が出力した静的サイト(`docs/_site`)を配る HTTP サーバで、そこに動的な経路を3種類だけ足している。



なぜ全部を Quarto に通さないか。成果物のページ(仕様書・WBS・一斉レビュー)は Markdown を正とし Quarto でレンダリングするが、バッチ実行状況ページだけは `serve_site.py` に埋め込んだ HTML 文字列(PIPELINE_PAGE)を直接配る。理由は2つ。

- ・ **更新頻度**: 実行中は秒単位で変わる。1回の再レンダリングに数十秒かかる Quarto を経由すると、そもそも「ライブ」にならない
- ・ **成果物ではない**: 実行状況は使い捨ての運転情報であって、リポジトリに残す文書ではない。Markdown を正とする必要がない

逆に一斉レビュー表は「レビュー対象の一覧」という**成果物**なので Markdown を正とし、操作性(フィルタ・検索・行内ボタン)はページ内 JS で後付けする、という分担にしている。

2 サーバを挟む理由(素の http.server を使わない)

表 2-1

理由	実装
プロジェクトごとに固定ポート	プロジェクト名の CRC32 から 8100-8899 を決定。複数プロジェクトが衝突しない
ローカル限定	既定で 127.0.0.1 にのみ bind(仕様書は社内資料)。LAN 公開は明示オプション+警告
キャッシュ無効	Cache-Control: no-store。再レンダリング後にリロードだけで最新が出る
MIME の確定	Windows のレジストリ由来 MIME は当てにならない(.js が text/plain になり Mermaid が動かない)ため上書き

理由	実装
書き込み経路の提供	承認・実行・裁定の POST(後述の安全設計つき)
EXE 配布	PyInstaller のエントリポイント。_site を同梱して単体実行ファイル化できる

3 GET

表 3-1

パス	返すもの	実装
/pipeline • /pipeline.html	バッチ実行状況ページ(HTML 定数)	PIPELINE_PAGE
/pipeline-status.json	ライブ進捗。クラッシュ残骸の補正つき	_mark_stale_status
/batch-queue?q=&chip=	残タスク一覧(実行順・検索・件数集計)	browser_run.batch_queue
/agent-logs/<fid>.txt	エージェント応答の全文	ファイル名を [\w.\-]+\..txt で検証してから配る
/favicon.ico	無ければ 204	404 でログを汚さないため
その他	docs/_site 配下の静的ファイル	SimpleHTTPRequestHandler

_mark_stale_status は、状態ファイルが `running` のまま残っている(実行プロセスが異常終了した)場合に、PID の死活を確認して `stopped` + 理由に読み替えて配る。ファイル自体は書き換えない読み取り専用の補正で、これが無いと画面だけが永遠に「実行中」に見える。

4 POST(すべて書き込み・実行系)

表 4-1

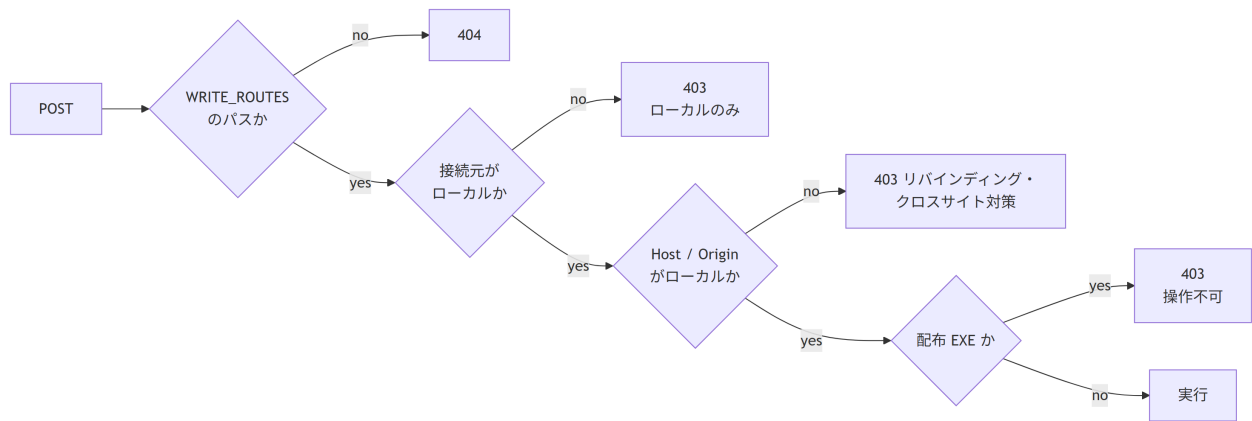
パス	用途	実装
/review-action	①②の承認 / 修正依頼、⑤の裁定	review_actions.approve / request_changes / adjudicate
/run-phase	①～⑤の単発実行	browser_run.start
/run-batch	連続実行の開始	browser_run.batch_start
/run-cancel	実行の中止(単発・連続とも)	browser_run.cancel
/run-skip	連続実行の「いまの 1 件」だけスキップ(バッチは続行)	browser_run.skip_current
/batch-priority	★ 優先の ON/OFF	browser_run.prioritize
/run-check	⑥完了検証(LLM 不使用・同期)	browser_run.run_check
/run-analyze	⑦分析の開始	browser_run.analyze_start

応答は常に {"ok": bool, "message": str}。ok: false は HTTP 422 で返し、message はそのまま画面に出す前提の日本語にする(「なぜできないか」が画面上で完結するように)。

5 安全設計

書き込み経路は次の 4 段で守る。

図 5-1



- 接続元 IP の確認だけでは足りない。(a) 攻撃者ドメインの DNS を `127.0.0.1` に向けて同一オリジンから fetch させる DNS リバインディング、(b) `text/plain` フォームで `preflight` を回避して JSON を送り込むクロスサイト POST、の 2 経路が残る。`Host` と `Origin` のホスト名までローカルであることを確認して塞ぐ
- `--host 0.0.0.0` で LAN 公開しても書き込みはローカル限定。閲覧は共有できるが、リモートから成果物を書き換え・実行させない
- 配布 EXE(FROZEN)では全 POST を 403。EXE の中身はビルド時点のスナップショットで、操作しても元プロジェクトに反映されないため、できないことを明示して返す
- 承認はサーバ側で機械レビューを再検証してから確定する。クライアントの表示 (ボタンが押せた=OK) を信用しない。「NG 付きの成果物を承認しない」という原則を UI の見た目ではなくサーバで強制する

6 2つの実装方式

表 6-1

方式	対象	更新のされ方
レンダ時に焼き込み	仕様書ページの 3 ウィジェット、WBS の⑥⑦ボタン、一斉レビュー表の行内ボタン	状態が変わったら再レンダリングが必要。操作後に <code>refresh_site</code> が自動で走る
ページ内で動的生成	バッチ実行状況ページの全要素	JSON ポーリングで秒単位に更新

焼き込み方式には「操作した瞬間には画面が変わらない」という制約があるため、**POST の成功後に `refresh_site`(WBS 再生成 + 一斉レビュー表再生成 + 差分レンダ)を サーバ側で実行し、完了後にページをリロード**する。この順序が重要で、先に「完了」を返すとポーリング側が古いページのままりロードしてしまう。

JS の実体は `browser_run.WIDGETS_JS` の 1 ファイルに集約し、`render_site.py` がレンダ後に `_site/lr-widgets.js` として書き出す。各ウィジェットの HTML は `<script src="/lr-widgets.js">` で参照する(ページごとに JS を複製しない)。公開オブジェクトは `lrRun / lrReview / lrAdj / lrCheck / lrAnalyze`。

7 出現条件の決定表

仕様書ページ(`/specs/F-xxxx.html`)は関数の「ホーム」として、そのときに可能な操作を **必ず 1 つだけ**出す。二重導線を作らないため、承認待ちの間は実行ボタンを出さない。

表 7-1

関数の状態	出るウィジェット	実装
① skeleton	「①を実行する」	<code>trigger_widget_html</code>
① draft(指摘なし)	承認ウィジェット(承認 / 修正依頼)	<code>widget_html(kind=spec)</code>
① draft(機械レビュー NG or 修正依頼あり)	承認ウィジェット + 再実行ボタン	同上 + <code>_rerun_wanted</code>
① reviewed・② なし	「②を実行する」	<code>trigger_widget_html</code>
② generated	承認ウィジェット(テスト仕様書)	<code>widget_html(kind=testspec)</code>
② approved・③ 未 freeze	「③を実行する」	<code>trigger_widget_html</code>
③ freeze 済み・④ 未実装	「④を実行する」	<code>trigger_widget_html</code>
④ 実装済み・⑤ 未 pass	「⑤を実行する」	<code>trigger_widget_html</code>
⑤ blocked(🔴)	裁定ウィジェット (ISSUE の質問を表示・回答して unblock)	<code>adjudicate_widget_html</code>
完了	何も出ない	—

WBS トップには 2 つ出る。どちらも**時期が来るまで表示しない**(押せないボタンを置かない)。

表 7-2

条件	ウィジェット
全関数の⑤が完了	「⑥完了検証を実行する」(LLM 不使用・数十秒で同期完了)
⑥が pass	「⑦分析を実行する」(定量計測 → 施策候補の提案まで。コードには触らない)

仕様書ページは3種のウィジェットの候補を持つが、出すのは高々1つで、優先順は「承認→実行→裁定」。承認待ちの間に実行ボタンを並べると、「承認すべきか作り直すべきか」が画面から読めなくなるため。

8 承認ウィジェットの構造

承認ウィジェットは4つの要素で構成する。

表 8-1

要素	内容	設計意図
機械レビュー結果	✓ 問題なし / ✗ 理由の全文(箇条書き)	件数だけ出して別ページを探させない
承認する	NGの間はグレースアウト。押してもサーバ側で再検証して弾く	NG 付き成果物を承認させない
修正依頼…	<code>review-feedback.md</code> に <code>pending</code> として追記	次に AI が①②を実行したとき自動で拾われる
再実行	NG か修正依頼が残るときだけ表示	押しても空振りするボタンを置かない

理由の全文をその場に置くのが要点。「一斉レビュー表に ✗ 4 件と出ているが中身が分からない」ため別ページを探しに行く、という往復を無くしている。

9 WBS(トップ)

`ledger.py wbs` が `docs/index.qmd` を生成する(手編集禁止)。200 関数を超えると自動でダッシュボードに切り替わるのが設計上の要点。

表 9-1

規模	トップページの構成
200 関数以下	進捗サマリ + コールグラフ + Open ISSUE + 全関数一覧
200 関数超	進捗サマリ + 要対応 + あなたへの質問 + 次の一手 + レガシーファイル別の進捗表。 全関数の明細は <code>docs/wbs/*.qmd</code> に分割

分割の理由は、2000 行の表を 1 ページに置くと「いま何が問題か」が読めなくなるため。ダッシュボード側には判断に必要な情報だけ(●・△・✕ と次の一手)を残す。

10 ① 一斉レビュー表

`review_checks.py` の `make_report()` が `docs/spec-review.md` を生成する。対象は `status: draft` の仕様書のみ。

並び順が優先度という設計にしている。

表 10-1

並び	意味	操作列
1	そのまま承認できる	承認 / 修正依頼…
2	承認できるが ISSUE(人への質問)がある	承認 / 修正依頼… + ISSUE リンク
3	機械レビュー NG(AI 修正待ち)	ボタンなし(「AI 修正待ち」と表示)

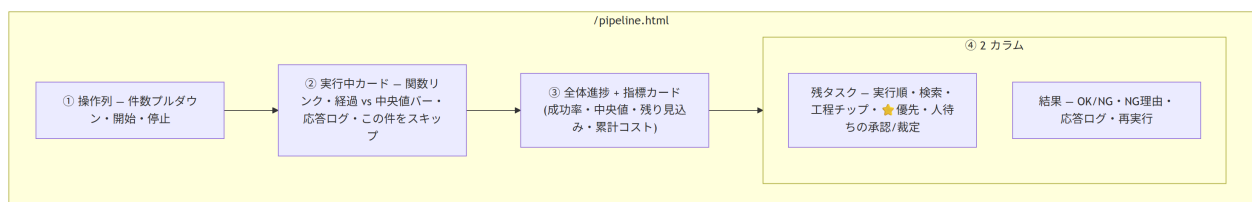
列は 6 つ — 関数 / 概要 / Confidence / 機械レビュー / 未確定(ISSUE) / 操作。Confidence は ●●● を 3 列に分けず「2●1●」の 1 列に統合した(列数を減らして 概要に幅を回すため)。

- 列幅は `table-layout: fixed` で固定する。Quarto 既定の内容比による自動配分だと、概要列が幅を占有して関数名が 1 文字ずつ縦に潰れる。余り幅は概要列だけに割り当てる
- 表に `min-width` を持たせ、画面が狭いときはフィルタバーを固定したまま表だけ横スクロールする
- フィルタチップ(すべて / 承認できる N / ISSUE あり N / AI 修正待ち N)と検索は ページ内 JS(`SPEC_REVIEW_PAGE_JS`)が表の直前に注入する。件数がそのまま 残作業量の指標になる
- 承認は `lrReview.approve` — 仕様書ページの承認ウィジェットと同じ経路を呼ぶ。UI が 2 つあってもサーバ側の検証・記録は 1 本

11 バッチ実行状況ページ

無人実行の運転席。`serve_site.py` の `PIPELINE_PAGE` が実体で、Quarto を通さない。

図 11-1



11.1 設計の要点

「上限件数」を数値入力からプルダウンにした。数値+プレースホルダでは何を 入れるべきか伝わらない。選択肢(全部(残り N 件) / お試しに 1 件だけ / 10 / 50 / 100) そのものが説明になり、「全部」に残件数が出ることで 1 日の作業量も見積もれる。

予算(\$)の上限入力は UI から外した。CLI の `--budget-usd` は残してあるが、画面の操作としては件数指定で足りる(コストは指標カードで見える)。

実行中カードの進捗バーは「その工程の中央値」を 100% とする。絶対時間では 速い/遅いの判断ができない。中央値超過で黄、2 倍超過で赤+「長引いています」と表示し、異常への気づきを色で伝える。

残タスクは既定で先頭 9 件しか出さない。2000 関数規模では全件表示が破綻するため、「次に何が来るか」だけを既定表示にし、それ以外は検索・チップで呼び出す設計にした。検索はサーバ側(`/batch-queue`)で行い、応答は 50 件で打ち切る(全件をブラウザに送らない)。

実行順の通し番号(今 / 次 1 / 次 2...)はサーバ側で振る。★優先で実行中の関数より 前に割り込んだ行が「次 0」になるなどのズレを防ぐため、クライアントでの補正はしない。

人待ち(承認・裁定)を同じ画面に置く。連続実行は承認待ち・裁定待ちを スキップし続けるので、人待ちこそが実際のボトルネックになる。バッチを回しながら この画面だけで詰まりを解消できるようにした。

11.2 残タスクの分類

`batch_queue()` は全関数を走査して、次の 2 系統に分類する。

表 11-1

種別	kind	判定
自動実行待ち	<code>spec / testspec / testcode / impl / test</code>	<code>_decide_kind(include_rerun=True)</code> が返す工程
人待ち	<code>approve-spec</code>	① が draft(かつ再実行対象でない)
人待ち	<code>approve-testspec</code>	② が generated
人待ち	<code>adjudicate</code>	<code>ledger.blocked_by</code> あり

自動実行待ちの並びは `_scan_targets` と同じ規則(★優先 → `functions.json` 順)で、画面の並び = 実際の実行順になるようにしている。

11.3 ポーリング設計

表 11-2

データ	間隔	理由
<code>/pipeline-status.json</code>	2.5 秒	小さい JSON。ライブ感が要る
<code>/batch-queue</code>	約 15 秒 + 検索時・操作後	全関数の frontmatter を読むので重い。ポーリングに載せない

結果パネルは内容が変わったときだけ再描画する。2.5 秒ごとに `innerHTML` を丸ごと差し替えると、人が読もうと開いた `<details>` が毎回閉じて「チカチカしてすぐ閉じる」状態になる(報告された不具合)。差分がある場合も、開いていた行は `data-rid` で復元する。

12 配布形態による機能差

`build_viewer.py` は `_site` を同梱した単体実行 EXE を作る(レビュアーへの配布用)。このとき画面は閲覧専用になる。

表 12-1

機能	<code>serve_site.py</code> で配信	配布 EXE
成果物・WBS・一斉レビュー表の閲覧	○	○
残タスク一覧(<code>/batch-queue</code>)	○	×
承認・修正依頼・裁定・実行(POST 全般)	○	×(403 と理由を返す)

EXE の中身はビルド時点のスナップショットで、操作しても元プロジェクトには反映されない。黙って失敗させず、理由を返して「できないこと」を明示する方針にしている。

状態ファイルと排他

画面が読み書きする実行時データは `.legacy-reverse/` に置く (git 管理外・消しても成果物は失われない)。

表 12-2

ファイル	書く人	読む人	形
<code>pipeline-status.json</code>	<code>RunStatus</code> (アトミック書き込み: tmp→replace)	<code>/pipeline-status.json</code>	状態・件数・指標・ <code>recent[]</code>
<code>batch-priority.json</code>	<code>/batch-priority</code>	<code>_scan_targets</code>	<code>{order: [fid...], retry: [fid...]}</code>
<code>agent-logs/F-xxxx.txt</code>	<code>save_agent_log</code> (追記)	<code>/agent-logs/*</code>	エージェント応答の全文
<code>pipeline-log.jsonl</code>	<code>log_line</code>	人・分析	1 行 1 試行
<code>run.lock</code>	ロック取得時	二重起動の判定	PID 入り

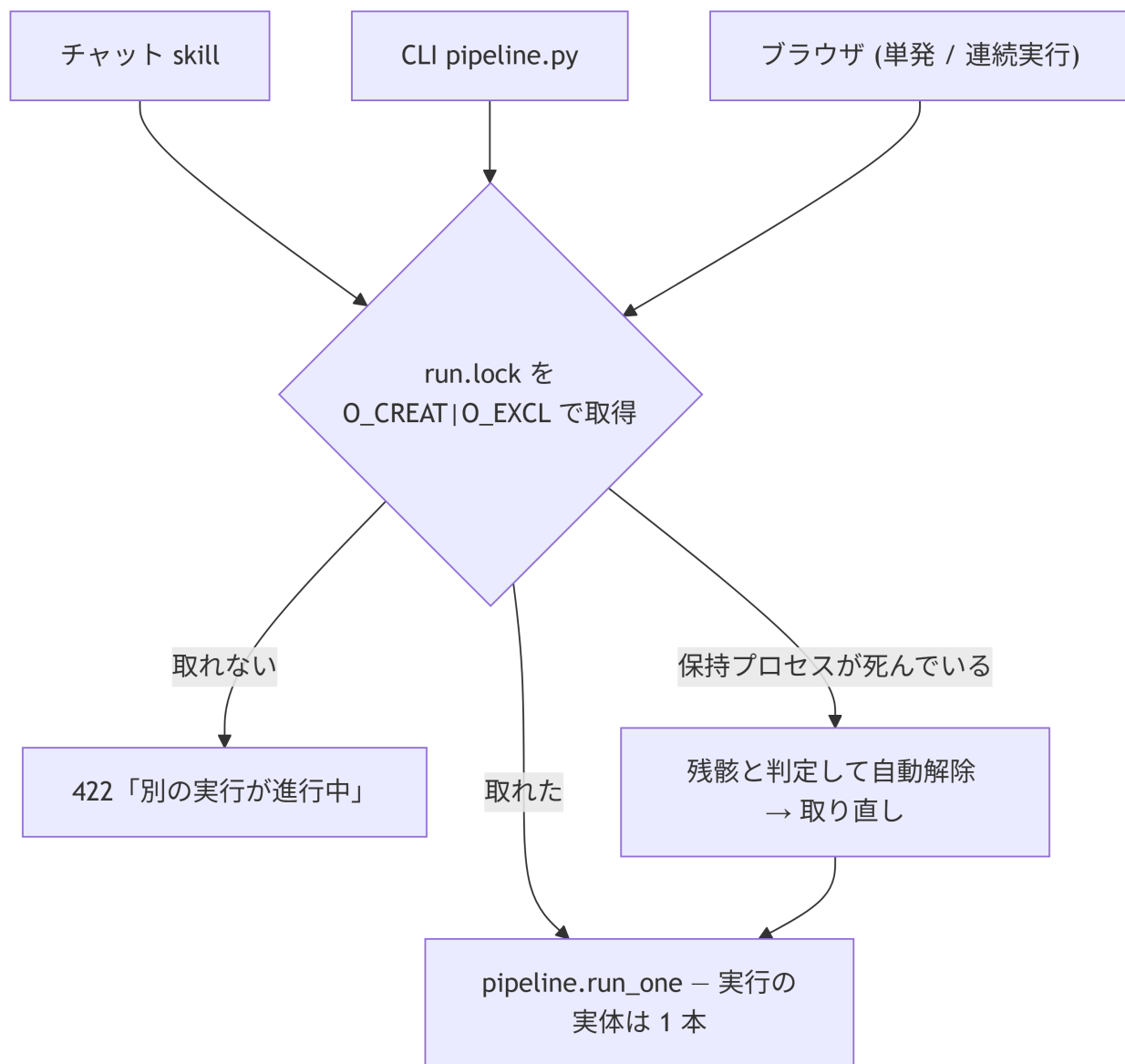
`recent[]` には工程(phase)を記録する。これが無いと結果パネルで「F-0019 が NG」までしか出せず、①なのか⑤なのかが分からない。

★優先の `retry` はワンショット。 `order` は実行順の割り込みだが、`retry` は「失敗スキップ集合からの解除」で、次の走査で消費される。毎回解除にすると、失敗し続ける関数が無限にリトライされてバッチが進まなくなる。

13 実行枠の排他

実行の入口は 3 つ(チャットの skill / CLI の無人バッチ / ブラウザ)あるが、同時に走ってよい実行は 1 つだけ。どの入口からでも同じロックを取り合う。

図 13-1



ロックは `O_CREAT|O_EXCL` の原子的なファイル作成で取る。状態ファイルの `state` を見るだけでは「チェックしてから状態を書くまでの隙間」で二重起動できるため、チェックと予約を 1 操作にする。ロックには PID を書き、取得失敗時に保持プロセスの死活を確認してクラッシュの残骸は自動解除する(人が手で消す運用にしない)。

POST はスレッドを起動して即座に応答を返す。実行完了まで HTTP 接続を保持しない (数分かかるため)。進捗は状態ファイル経由でポーリングして受け取る。

中止・スキップの2系統

連続実行には「止める」操作が2つあり、意味が違う。

表 13-1

操作	エンドポイント	効果	使う場面
停止	<code>/run-cancel</code>	バッチ全体を止める	もう今日は終わり
この件をスキップ	<code>/run-skip</code>	いまの1件だけ中止し、次へ進む	1件が長引いている

実装は `_AnyEvent`(複数 Event の OR ビュー)で、`run_one` には「バッチ全体の中止」と「この1件のスキップ」を OR にしたものを渡す。どちらが立っても実行中の headless プロセスは止まるが、**バッチのループを抜けるかどうか**が呼び出し側で分かれる。

スキップされた(関数, 工程)は失敗と同じ扱いで、そのバッチ内では再試行されない。ただし状態としては未着手のままなので、★優先で拾い直せるし、次回バッチでは自動的に対象へ戻る。

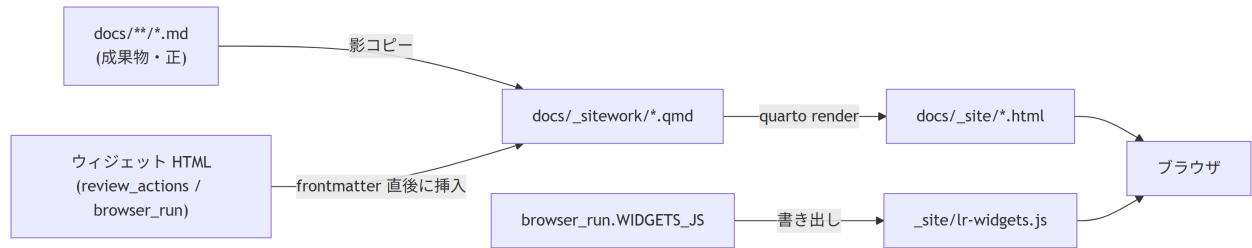
💡 安全停止

連続3件失敗するとバッチ全体が停止する。API キー切れ・レート枯渇などの環境異常で残り全件を「失敗」として消化してしまうのを防ぐため。人がスキップした分はこの連続失敗カウントに含めない(人の操作は環境異常ではない)。

レンダリング経路

成果物(.md)がブラウザに出るまでの流れ。

図 13-2



なぜ影コピーを作るか。 Mermaid は `.qmd` でしか描画されない(`.md` のままだと 図がソースのまま出る)。成果物には GitHub 流の ```mermaid` で書かせ、影コピー生成時に ```mermaid` へ変換する。 `quarto render docs` を直接叩くと図が出ないのはこのため。

差分レンダリングが既定。変わったページだけを出し直すので 2 回目以降は数十秒で済む。ただし検索索引は更新されないため、節目で `--full` を 1 回実行する運用にしている。

表 13-2

判断	採用した理由	捨てた案
バッチ画面だけ Quarto を通さない	秒単位の更新が要る。成果物ではない	全画面 Quarto(ライブにならない)
ウィジェットをレンダ時に焼き込む	静的サイトのまま配れる。EXE 配布と両立	全画面を SPA 化(配布・オフライン閲覧が壊れる)
承認 UI を複数画面に置く	一覧からも詳細からも承認したい	単一画面に限定(往復が増える)
承認の検証はサーバ側で再実行	クライアント表示を信用しない	ボタンの活性制御だけ(改竄・競合に弱い)
残タスクは既定 9 件 + 検索	2000 関数で破綻しない	全件表示 + クライアント検索(重い)
実行順番号をサーバで振る	画面の並び = 実際の実行順を保証	クライアント補正(ズレる)
人待ちを運転席に集約	実際のボトルネックだから	WBS へ戻らせる(往復)

関連: 運用設計 — 導入・日常運用・配布 / 品質ゲート仕様 — 承認前に機械が何を検証するか